



Introduction to Programming in C++ Eighth Edition

Chapter 9: Value-Returning Functions

Objectives

- Raise a number to a power using the `pow` function
- Return the square root of a number using the `sqrt` function
- Generate random numbers
- Create and invoke a function that returns a value
- Pass information *by value* to a function
- Write a function prototype
- Understand a variable's scope and lifetime

Functions

- A function is a block of code that performs a task
- Every C++ program contains at least one function (main)
 - Most contain many functions
- Some functions are built-in functions (part of C++): defined in language libraries
- Others, called program-defined functions, are written by programmers; defined in a program
- Functions allow for blocks of code to be used many times in a program without having to duplicate code

Functions (cont'd.)

- Functions also allow large, complex programs to be broken down into small, manageable sub-tasks
- Each sub-task is solved by a function, and thus different people can write different functions
- Many functions can then be combined into a single program
- Typically, `main` is used to call other functions, but any function can call any other function

Functions (cont'd.)

Illustration A



Helen:

1. ask ticket agent for a senior ticket
2. give ticket agent \$5
3. receive senior ticket from ticket agent

Ticket agent (value-returning function):

1. take \$5 from Helen
2. give Helen a senior ticket

Illustration B



Helen:

1. tell Penelope to have fun playing games
2. give Penelope \$5

Penelope (void function):

1. take \$5 from Helen
2. buy game tickets with the \$5
3. play games and have fun

Figure 9-1 Illustrations of value-returning and void functions

Value-Returning Functions

- All functions are either value-returning or void
- All **value-returning functions** perform a task and then return precisely one value
- In most cases, the value is returned to the statement that called the function
- Typically, a statement that calls a function assigns the return value to a variable
 - However, a return value could also be used in a comparison or calculation or could be printed to the screen

The `pow` Function

- The **`pow` function** is a convenient tool to raise a number to a power (**exponentiation**)
- The `pow` function raises a number to a power and returns the result as a `double` number
- Syntax is `pow(x, y)`, in which `x` is the base and `y` is the exponent
- At least one of the two arguments must be a `double`
- Program must contain the `#include <cmath>` directive to use the `pow` function

The `pow` Function (cont'd.)

How To Use the `pow` Function

Syntax

`pow(x, y)`

requires the `#include <cmath>` directive

Example 1

```
double cube = 0.0;  
cube = pow(4.0, 3);
```

The assignment statement assigns the number 64.0, which is 4.0 raised to the third power, to the `cube` variable.

Example 2

```
cout << pow(100, 0.5);
```

The statement displays the number 10, which is 100 raised to the 0.5 power. The `pow(100, 0.5)` expression is equivalent to finding the square root of the number 100.

Example 3

```
double area = 0.0;  
double radius = 5.0;  
area = 3.14 * pow(radius, 2);
```

The assignment statement raises the value stored in the `radius` variable to the second power, giving 25.0. It then multiplies the 25.0 by 3.14 and assigns the product (78.5) to the `area` variable.

Figure 9-2 How to use the `pow` function

The `sqrt` Function

- **`sqrt` function** is a built-in value-returning function that returns a number's square root as a `double`
- Definition contained in `cmath` library
 - Program must contain `#include <cmath>` to use it
- Syntax: `sqrt(x)`, in which `x` is a `double` or `float`
 - Here, `x` is an **actual argument**, which is an item of information a function needs to perform its task
- Actual arguments are passed to a function when called

The `sqrt` Function (cont'd.)

How To Use the `sqrt` Function

Syntax

`sqrt(x)`

requires the `#include <cmath>` directive

Example 1

```
double squareRoot = 0.0;  
squareRoot = sqrt(100.0);
```

The `sqrt` function finds the square root of the `double` number 100.0 and then returns the result (the `double` number 10.0) to the assignment statement, which assigns the return value to the `squareRoot` variable.

Example 2

```
double num = 0.0;  
cout << "Enter a number: ";  
cin >> num;  
cout << sqrt(num);
```

The `sqrt` function finds the square root of the `double` number stored in the `num` variable and then returns the result to the `cout` statement, which displays the return value on the computer screen.

Figure 9-3 How to use the `sqrt` function

The Hypotenuse Program

- Program that calculates and displays the length of a right triangle hypotenuse
- Program uses Pythagorean theorem
 - Requires squaring and taking square root
- `pow` function can be used to square
- `sqrt` function can be used to take square root
- Both are built-in value-returning functions

The Hypotenuse Program (cont'd.)

Pythagorean theorem

The theorem states that the length of a right triangle's hypotenuse is equal to the square root of the sum of the squares of the lengths of the triangle's two adjacent sides. In other words, the hypotenuse's length is equal to the square root of the following sum: $(\text{side } a \text{ length})^2 + (\text{side } b \text{ length})^2$.

Example: side a length is 10 and side b length is 24

- | | | |
|--|-------------------|--------------------------|
| 1. square side a length | $10 * 10 = 100$ | |
| 2. square side b length | $24 * 24 = 576$ | |
| 3. sum the squares from Steps 1 and 2 | $100 + 576 = 676$ | |
| 4. find the square root of the sum from Step 3 | 26 | length of the hypotenuse |

Figure 9-4 Pythagorean theorem

The Hypotenuse Program (cont'd.)

Problem specification

Create a program that uses the Pythagorean theorem to calculate the length of a right triangle's hypotenuse, given the lengths of its two adjacent sides (*side a* and *side b*). Display the length with one decimal place.

IPO chart information

Input

side a length
side b length

Processing

Calculate the hypotenuse length

Output

Hypotenuse length

Algorithm

1. Enter side a length and side b length

2. Calculate the hypotenuse length using the Pythagorean theorem:

$$\text{hypotenuse} = \sqrt{\text{side a length}^2 + \text{side b length}^2}$$

C++ instructions

```
double sideA = 0.0;
double sideB = 0.0;
```

```
double sumSqr = 0.0;
```

```
double hypotenuse = 0.0;
```

```
cout << "Side a length: ";
cin >> sideA;
cout << "Side b length: ";
cin >> sideB;
```

```
sumSqr = pow(sideA, 2) +
pow(sideB, 2);
hypotenuse = sqrt(sumSqr);
```

```
cout << "Hypotenuse length: "
<< hypotenuse << endl;
```

Figure 9-5 Problem specification, IPO chart information, and C++ instructions for the Hypotenuse Program

The Hypotenuse Program (cont'd.)

```
1 //Hypotenuse.cpp - displays the length of a right
2 //triangle's hypotenuse
3 //Created/revised by <your name> on <current date>
4
5 #include <iostream>
6 #include <iomanip>
7 #include <cmath>
8 using namespace std;
9
10 int main()
11 {
12     double sideA = 0.0;
13     double sideB = 0.0;
14     double sumSqr = 0.0;
15     double hypotenuse = 0.0;
16
17     //get lengths of two sides
```

required for the pow
and sqrt functions

Figure 9-6 Beginning of Hypotenuse program

The Hypotenuse Program (cont'd.)

```
18  cout << "Side a length: ";
19  cin >> sideA;
20  cout << "Side b length: ";
21  cin >> sideB;
22
23  //calculate the hypotenuse length
24  sumSqr = pow(sideA, 2) + pow(sideB, 2);
25  hypotenuse = sqrt(sumSqr);
26
27  //display the hypotenuse length
28  cout << fixed << setprecision(1);
29  cout << "Hypotenuse length: " << hypotenuse << endl;
30  return 0;
31 }
```

use the pow and
sqrt functions

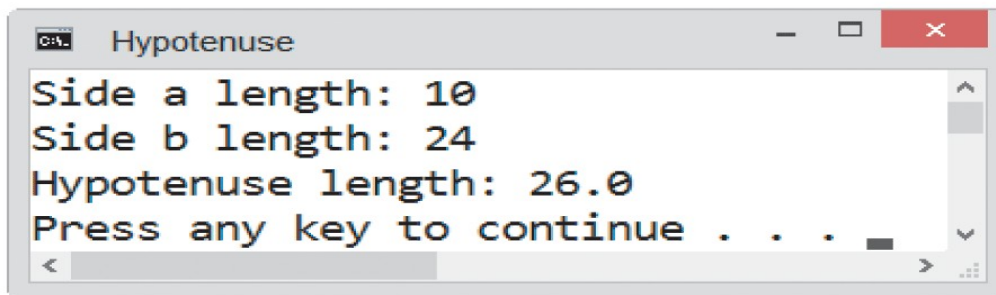


Figure 9-6 Completion of Hypotenuse Program and sample run

The `rand`, `srand`, and `time` Functions

- C++ provides a pseudo-random number generator
 - Produces a sequence of numbers that meet certain statistical requirements for randomness
 - Numbers chosen uniformly from finite set of numbers
 - Not truly random but sufficient for practical purposes
- Random number generator in C++: `rand` function
 - Returns an integer between 0 and `RAND_MAX`, inclusive
 - `RAND_MAX` is a built-in constant (≥ 32767)

The `rand`, `srand`, and `time` Functions (cont'd.)

- `rand` function's syntax: `rand()`
 - Doesn't require any actual arguments, but parentheses are still required
- Expression:
$$\text{lowerBound} + \text{rand}() \% (\text{upperBound} - \text{lowerBound} + 1)$$
 - Allows ranges other than 0 to `RAND_MAX` to be used
 - Range is *upperBound* to *lowerBound*
- Initialize random number generator each time
 - Otherwise, will produce the same sequence

The `rand`, `srand`, and `time` Functions (cont'd.)

How To Use the `rand` Function

Syntax

`rand()`

your compiler may require the
`#include <cstdlib>` directive

Example 1

```
int randomNum = 0;  
randomNum = rand();
```

The `rand` function generates a random integer that is greater than or equal to 0 but less than or equal to `RAND_MAX`. It then returns the random integer to the assignment statement, which assigns it to the `randomNum` variable.

Example 2

```
cout << rand();
```

The `rand` function generates a random integer that is greater than or equal to 0 but less than or equal to `RAND_MAX`. It then returns the random integer to the `cout` statement, which displays it on the computer screen.

Example 3

```
int tripleNum = 0;  
tripleNum = rand() * 3;
```

The `rand` function generates a random integer that is greater than or equal to 0 but less than or equal to `RAND_MAX`. It then returns the random integer to the assignment statement, which multiplies it by 3 and assigns the result to the `tripleNum` variable.

Figure 9-7 How to use the `rand` function

The `rand`, `srand`, and `time` Functions (cont'd.)

How To Generate Random Integers within a Specific Range

Syntax

lowerBound + **`rand()`** % (*upperBound* – *lowerBound* + **1**)

Example 1

```
cout << 1 + rand() % (6 - 1 + 1);
```

displays a random integer from 1 through 6 on the computer screen

Figure 9-8 How to generate random integers within a specific range

The rand, srand, and time Functions (cont'd.)

Results using different rand values:

rand value: 27

6 - 1 + 1 is evaluated first and results in 6

27 % 6 is evaluated next and results in 3

1 + 3 is evaluated last and results in 4

1 + 27 % (6 - 1 + 1)

1 + 27 % 6

1 + 3

4

rand value: 8

6 - 1 + 1 is evaluated first and results in 6

8 % 6 is evaluated next and results in 2

1 + 2 is evaluated last and results in 3

1 + 8 % (6 - 1 + 1)

1 + 8 % 6

1 + 2

3

rand value: 324

6 - 1 + 1 is evaluated first and results in 6

324 % 6 is evaluated next and results in 0

1 + 0 is evaluated last and results in 1

1 + 324 % (6 - 1 + 1)

1 + 324 % 6

1 + 0

1

Example 2

```
int num = 0;
```

```
num = 10 + rand() % (100 - 10 + 1);
```

assigns a random integer from 10 through 100 to the num variable

Results using different rand values:

rand value: 352

100 - 10 + 1 is evaluated first and results in 91

352 % 91 is evaluated next and results in 79

10 + 79 is evaluated last and results in 89

10 + 352 % (100 - 10 + 1)

10 + 352 % 91

10 + 79

89

rand value: 4

100 - 10 + 1 is evaluated first and results in 91

4 % 91 is evaluated next and results in 4

10 + 4 is evaluated last and results in 14

10 + 4 % (100 - 10 + 1)

10 + 4 % 91

10 + 4

14

rand value: 2500

100 - 10 + 1 is evaluated first and results in 91

2500 % 91 is evaluated next and results in 43

10 + 43 is evaluated last and results in 53

10 + 2500 % (100 - 10 + 1)

10 + 2500 % 91

10 + 43

53

Figure 9-8 How to generate random integers within a specific range

The `rand`, `srand`, and `time` Functions (cont'd.)

- Use `srand` function (a void function) to initialize random number generator
- Syntax: `srand(seed)`, in which `seed` is an integer actual argument that represents the starting point of the generator
 - Commonly initialized using the `time` function
 - Ensures unique sequence of numbers for each program run

The `rand`, `srand`, and `time` Functions (cont'd.)

- **`time` function** is a value-returning function that returns current time in number of seconds since January 1, 1970
 - Returns a `time_t` object, so must be cast to an integer before passing to `srand`
 - Program must contain `#include <ctime>` directive to use it

The rand, srand, and time Functions (cont'd.)

How To Use the srand Function

Syntax

srand(seed)

your compiler may require the
#include <cstdlib> directive

Example 1

```
int x = 0;
cout << "Enter an integer: ";
cin >> x;
srand(x);
cout << rand() << endl;
```

The srand function initializes the random number generator using the integer entered by the user. The cout statement displays a random integer that will be greater than or equal to 0 but less than or equal to RAND_MAX.

Example 2

```
srand(static_cast<int>(time(0)));
cout << rand() << endl;
```

the time function requires the
#include <ctime> directive

The srand function initializes the random number generator using the value returned by the time function after it has been converted to the int data type. The cout statement displays a random integer that will be greater than or equal to 0 but less than or equal to RAND_MAX.

Example 3

```
int randNum = 0;
srand(static_cast<int>(time(0)));
randNum = 1 + rand() % (10 - 1 + 1);
```

the time function requires the
#include <ctime> directive

The srand function initializes the random number generator using the value returned by the time function after it has been converted to the int data type. The assignment statement assigns a random integer that is greater than or equal to 1 but less than or equal to 10 to the randNum variable.

Figure 9-9 How to use the srand function

The Guessing Game Program

- Program generates a random number from 1 to 10 and then allows the user as many choices as needed to guess the number.
- The `srand`, `time`, and `rand` functions are all utilized in the program.

The Guessing Game Program(cont'd.)

Problem specification

Create a program that generates a random number from 1 through 10 and then allows the user as many chances as needed to guess the number. If the user guesses the number, display the message "Yes, the number is" followed by the number. Otherwise, display the "Sorry, guess again:" message.

IPO chart information

Input

random number (from
1 through 10)
guess

C++ instructions

```
int randomNumber = 0;

int numberGuess = 0;
```

Processing

none

Output

a r o r ate message

Algorithm

1 generate random number

enter guess

repeat the (guess is not
random number)
do a r o r
again message
enter guess
end repeat

do a r o r
s and the random number

```
srand(static_cast<int>(time(0)));
randomNumber = 1 + rand() % (10 - 1 + 1);

cout << "Guess a number from 1 through 10: ";
cin >> numberGuess;

while (numberGuess != randomNumber)
{
    cout << "Sorry, guess again: ";
    cin >> numberGuess;
} //end while

cout << endl << "Yes, the number is "
<< randomNumber << "." << endl;
```

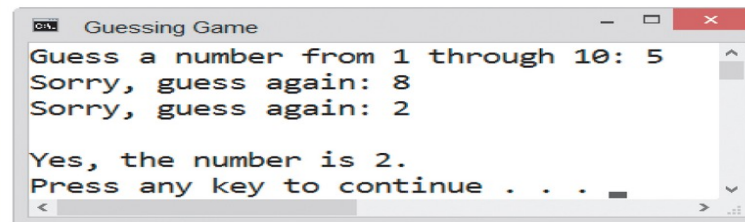
Figure 9-10 Problem specification, IPO chart information, and C++ instructions for the Guessing Game Program

The Guessing Game Program(cont'd.)

```
1 //Guessing Game.cpp - number guessing game
2 //Created/revised by <your name> on <current date>
3
4 #include <iostream>
5 #include <ctime>
6 // #include <cstdlib>
7 using namespace std;
8
9 int main()
10 {
11     int randomNumber = 0;
12     int numberGuess = 0;
13
14     //generate a random number from 1 through 10
15     srand(static_cast<int>(time(0)));
16     randomNumber = 1 + rand() % (10 - 1 + 1);
17
18     //get first guess from user
19     cout << "Guess a number from 1 through 10: ";
20     cin >> numberGuess;
21
22     while (numberGuess != randomNumber)
23     {
24         cout << "Sorry, guess again: ";
25         cin >> numberGuess;
26     } //end while
27
28     cout << endl << "Yes, the number is "
29         << randomNumber << "." << endl;
30     return 0;
31 } //end of main function
```

required for the
time function

your compiler may require
this directive to use the
rand and srand functions



```
Guessing Game
Guess a number from 1 through 10: 5
Sorry, guess again: 8
Sorry, guess again: 2
Yes, the number is 2.
Press any key to continue . . .
```

Figure 9-11 Guessing Game Program and sample run

Creating Program-Defined Value-Returning Functions

- A program-defined value-returning function definition is composed of a header and a body
- Header (first line) contains return data type, name of function, and an optional *parameterList*
 - Rules for function names are same as for variables
 - Good idea to use meaningful names that describe function's purpose
 - Memory locations in *parameterList* are called **formal parameters**
 - Each stores an item of information passed to the function when it is called

Creating Program-Defined Value-Returning Functions (cont'd.)

- Function body contains instructions for performing the function's assigned task
- Surrounded by braces ({ })
- Last statement is usually the **return statement**
 - Returns one value (must match return data type in function header)
- After `return` statement is processed, program execution continues in calling function
- Good idea to include comment (such as `//end of functionName`) to mark end of function

Creating Program-Defined Value-Returning Functions (cont'd.)

How To Create a Program-Defined Value-Returning Function

Syntax

```
returnDataType functionName([parameterList])  
{  
    one or more statements  
    return expression;  
} //end of functionName function
```

The diagram illustrates the syntax of a program-defined value-returning function. The function header, consisting of the return type, function name, and parameter list in parentheses, is enclosed in a box and labeled 'function header'. The function body, which is enclosed in curly braces and contains one or more statements followed by a return statement, is enclosed in a box and labeled 'function body'. The closing brace of the function body is followed by a comment indicating the end of the function.

Figure 9-12 How to create a program-defined value-returning function

Creating Program-Defined Value-Returning Functions (cont'd.)

Example 1

```
int getRandomNumber()
{
    int randInteger = 0;
    randInteger = 1 + rand() % (10 - 1 + 1);
    return randInteger;
} //end of getRandomNumber function
```

function definition

The function generates a random integer from 1 through 10 and then returns the random integer.

Example 2

```
double getRectangleArea(double len, double wid)
{
    return len * wid;
} //end of getRectangleArea function
```

The function calculates the area of a rectangle and then returns the result as a double number.

Example 3

```
double getSubtotal(int sold, double costPerItem)
{
    double subtotal = 0.0;
    subtotal = sold * costPerItem;
    return subtotal;
} //end of getSubtotal function
```

The function calculates the subtotal and then returns the result as a double number.

Figure 9-12 How to create a program-defined value-returning function

Calling a Function

- A function must be called (invoked) to perform its task
- `main` is automatically called when program is run
- Other functions must be called by a statement
- Syntax for calling a function:

functionName (*argumentList*) ;

- *argumentList* is list of actual arguments (if any)
- An actual argument can be a variable, named constant, literal constant, or keyword

Calling a Function (cont'd.)

- Value-returning functions are typically called from statements that:
 - Assign the return value to a variable
 - Use the return value in a calculation or comparison
 - Display the return value
- A call to a void function is an independent statement because void functions do not return values

Calling a Function (cont'd.)

- C++ allows you to pass either a variable's value or its address to a function
- Passing a variable's value is referred to as **passing *by value***
- Passing a variable's address is referred to as **passing *by reference***
- Default is passing *by value*
- Number, data type, and ordering of actual arguments must match the formal parameters in function header
 - Names do not need to match (different names are better)

Calling a Function (cont'd.)

How To Call (Invoke) a Value-Returning Function

Syntax

`functionName([argumentList])`

Example 1

```
int num1 = 0;  
num1 = getRandomNumber();
```

The assignment statement calls the `getRandomNumber` function and then assigns the function's return value to the `num1` variable.

Example 2

```
cout << getRectangleArea(7.25, 21.0);
```

The `cout` statement calls the `getRectangleArea` function, passing it the `double` numbers 7.25 and 21.0. It then displays the function's return value on the computer screen.

Example 3

```
int quantity = 0;  
double itemPrice = 0.0;  
double salesTax = 0.0;  
cin >> quantity;  
cin >> itemPrice;  
salesTax = getSubtotal(quantity, itemPrice) * 0.05;
```

The assignment statement calls the `getSubtotal` function, passing it the integer stored in the `quantity` variable and the `double` number stored in the `itemPrice` variable. It then multiplies the function's return value by the `double` number 0.05 and stores the result in the `salesTax` variable.

Figure 9-13 How to call (invoke) a value-returning function

Calling a Function (cont'd.)

```
salesTax = getSubtotal(quantity, itemPrice) * 0.05;  
           ↓           ↓  
double getSubtotal(int sold, double costPerItem)  
{  
    double subtotal = 0.0;  
    subtotal = sold * costPerItem;  
    return subtotal;  
} //end of getSubtotal function
```

Figure 9-14 Function call and function definition

The Savings Account Program

- The program allows the user to enter the initial deposit made into a savings account and the annual interest rate.
- The program displays the amount of money in the account at the end of 1 through 3 years, assuming no additional deposits or withdrawals are made.

The Savings Account Program (cont'd.)

Problem specification

Create a program that allows the user to enter the initial deposit made into a savings account and the annual interest rate. The program should display the amount of money in the account at the end of 1 through 3 years, assuming no additional deposits or withdrawals are made. You can calculate the savings account balances using the following formula: $b = p * (1 + r)^n$. In the formula, p is the principal (the amount of the initial deposit), r is the annual interest rate, n is the number of years, and b is the balance in the savings account at the end of the n^{th} year.

Example 1: using 1000 for p , 0.02 for r , and 1 for n

$b = 1000 * (1 + 0.02)^1$

$b = 1020$

Example 2: using 5000 for p , 0.03 for r , and 2 for n

$b = 5000 * (1 + 0.03)^2$

$b = 5304.50$

main function

IPO chart information

Input

de osit
rate
ear counter to

Processing

none

Output

account balance at end of each of
the ears

Algorithm

enter de osit and rate

re eat for ear fro to

call the et alance function to
calculate the current account balance
ass the de osit rate and ear

dis la the ear and current
account balance

end re eat

main function

C++ instructions

```
int deposit = 0;
double interestRate = 0.0;
this variable is created and initialized
in the for clause
```

```
double acctBalance = 0.0;
```

```
cout << "Deposit: ";
cin >> deposit;
cout << "Rate (in decimal form): ";
cin >> interestRate;
for (int year = 1; year < 4; year += 1)
{
    acctBalance = getBalance(deposit,
    interestRate, year);

    cout << "Year " << year << ": $"
    << acctBalance << endl;
} //end for
```

Figure 9-15 Problem specification, IPO chart information, and C++ instructions for the Savings Account Program

The Savings Account Program (cont'd.)

getBalance function IPO chart information

Input

de osit for al ara eter
rate for al ara eter
ear for al ara eter

Processing

none

Output

account balance

Algorithm

calculate the account balance
return the account balance

getBalance function C++ instructions

```
int amount  
double rate  
int y
```

```
double balance = 0.0;
```

```
balance = amount * pow((1 + rate), y);  
return balance;
```

Figure 9-15 Problem specification, IPO chart information,
and C++ instructions for the Savings Account Program

The Savings Account Program (cont'd.)

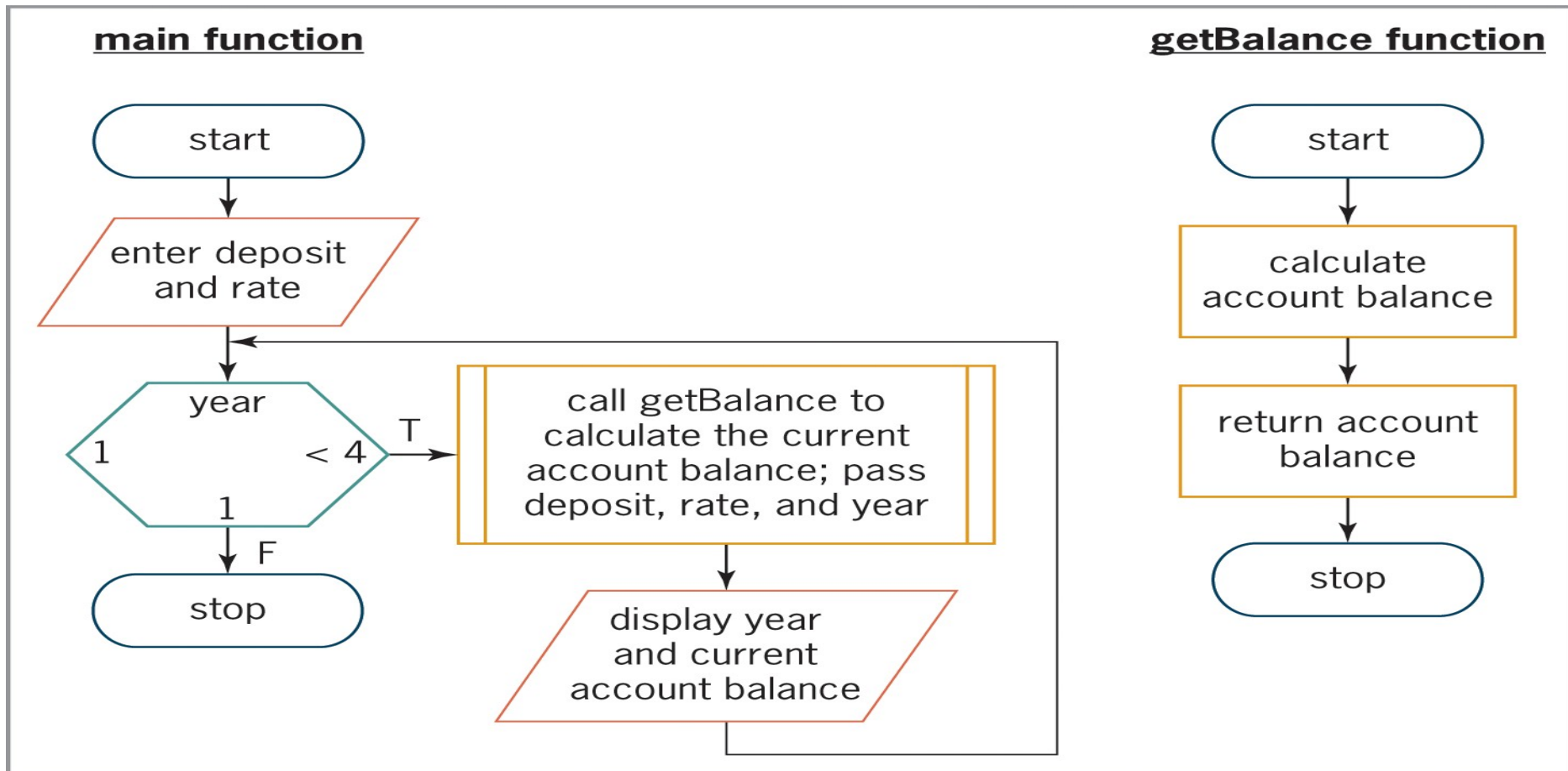


Figure 9-16 Flowcharts for the Savings Account Program

Function Prototypes

- When a function definition appears below the `main` function, you must enter a function prototype above the `main` function
- A **function prototype** is a statement that specifies the function's name, data type of its return value, and data type of each of its formal parameters (if any)
 - Names for the formal parameters are not required
- Programmers usually place function prototypes at beginning of program, after the `#include` directives

Function Prototypes (cont'd.)

How To Write a Function Prototype

Syntax

returnDataType *functionName*([*parameterList*]);

semicolon

each formal parameter's data type and (optionally) name

Example 1 – with the optional names

```
double getBalance(int amount, double rate, int y);
```

Example 2 – without the optional names

```
double getBalance(int, double, int);
```

Figure 9-17 How to write a function prototype

Completing the Savings Account Program

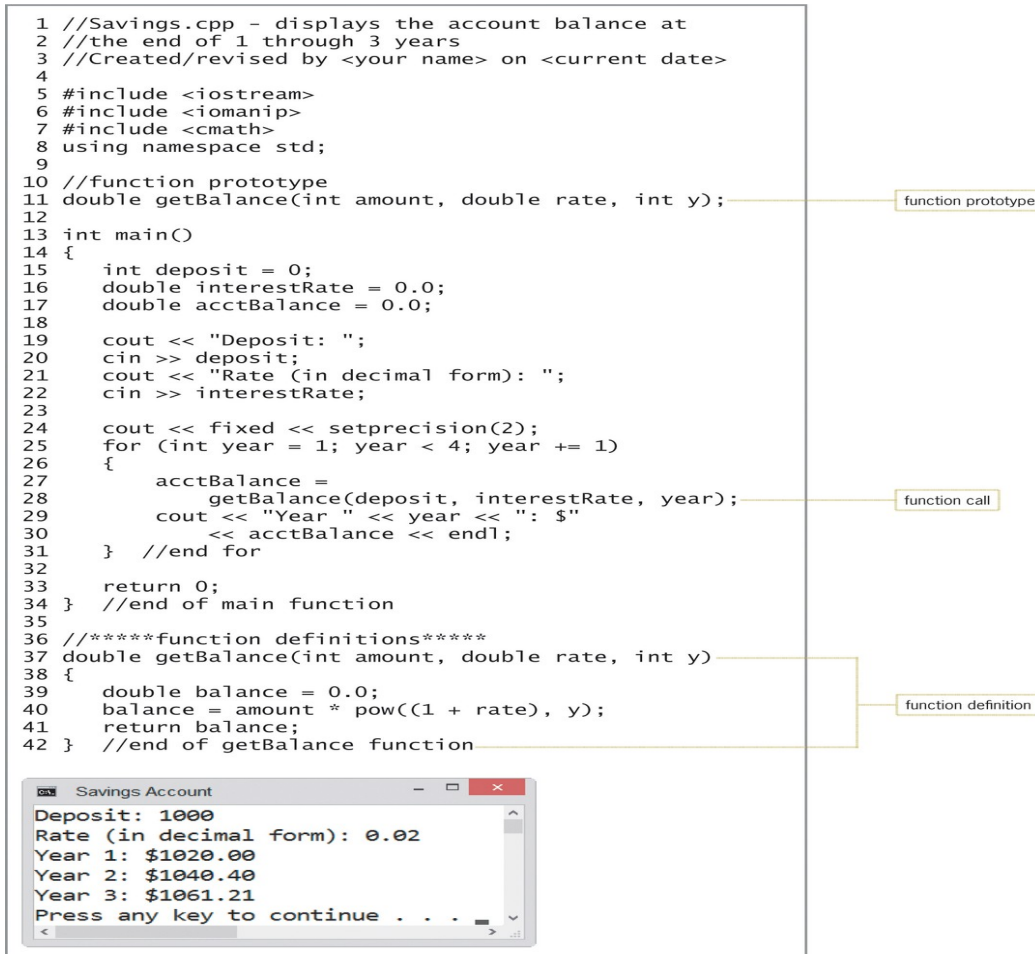


Figure 9-18 Savings Account Program

The Scope and Lifetime of a Variable

- A variable's **scope** indicates where in the program the variable can be used
- A variable's **lifetime** indicates how long the variable remains in the computer's internal memory
- Both scope and lifetime are determined by where you declare the variable in the program
- Variables declared within a function and those that appear in a function's *parameterList* have a local scope and are referred to as local variables

The Scope and Lifetime of a Variable (cont'd.)

- **Local variables** can be used only by the function in which they are declared or in whose *parameterList* they appear
 - Remain in internal memory until the function ends
- **Global variables** are declared outside of any function in the program
 - Remain in memory until the program ends
- Any statement can use a global variable

The Scope and Lifetime of a Variable (cont'd.)

- Declaring a variable as global can allow unintentional errors to occur
 - e.g., a function that should not have access to the variable inadvertently changes the variable's contents
- You should avoid using global variables unless necessary
- If more than one function needs to access the same variable, it is better to create a local variable in one function and pass it to other functions that need it

Summary

- Functions
 - Allow programmers to avoid duplicating code
 - Allow for large, complex programs to be broken into small, manageable tasks
- Some functions are built into the language, and others are program-defined
- All functions are either value-returning or void
- A value-returning function returns one value
 - Value returned to statement that called the function
- A void function returns no value

Summary (cont'd.)

- Use the `sqrt` function to find the square root of a number
- Use the `pow` function to raise a number to a power
- Items in parentheses in a function call are called actual arguments
- The `rand` function is used to generate random numbers
 - Returns an integer between 0 and `RAND_MAX`
- `srand` function is used to initialize `rand` function
 - `time` function usually used as seed (starting point)

Summary (cont'd.)

- Function definition composed of header and body
- Header specifies function name, return data type, and formal parameter names and types (if any)
 - Data types and ordering of formal parameters must match data types and ordering of actual arguments
- Body contains instructions for performing the function's assigned task
 - Surrounded by braces ({ })
- `return` statement returns the result of an expression to the calling function

Summary (cont'd.)

- You call a function by including its name and actual arguments (if any) in a statement
- Variables in C++ are passed *by value* by default
- A function prototype must be provided for each function defined below the `main` function
- Scope of a variable indicates where in the program it can be used
- Lifetime of a variable indicates how long it will stay in internal memory

Summary (cont'd.)

- Local variables can be used only within the function in which they are declared or in whose *parameterList* they appear
 - Remain in memory until the function ends
- Global variables can be used anywhere
 - Remain in memory until the program ends
- If more than one memory location have the same name, position of the statement in which the name is used determines which location is used

Lab 9-1: Stop and Analyze

- Study the program in Figure 9-26, and then answer the questions

```
1 //Lab9-1.cpp – circle calculations
2 //Created/revised by <your name> on <current date>
3
4 #include <iostream>
5 #include <cmath>
6 using namespace std;
7
8 //function prototypes
9 double getArea(double rad);
10 double getDiameter(double rad);
11
```

Figure 9-26 Code for Lab 9-1

Lab 9-1: Stop and Analyze (cont'd.)

```
12 int main()
13 {
14     int choice = 0;
15     double radius = 0.0;
16
17     cout << "1    Circle area" << endl;
18     cout << "2    Circle diameter" << endl;
19     cout << "Enter your choice (1 or 2): ";
20     cin >> choice;
21
22     if (choice < 1 || choice > 2)
23         cout << "Invalid choice" << endl;
24     else
25     {
26         cout << "Radius: ";
27         cin >> radius;
28         if (choice == 1)
29             cout << "Area: " << getArea(radius);
30         else
31             cout << "Diameter: " << getDiameter(radius);
32         //end if
33         cout << endl;
34     } //end if
35     return 0;
36 } //end of main function
37
38 /******function definitions*****/
39 double getArea(double rad)
40 {
41     const double PI = 3.141593;
42     double area = 0.0;
43     area = PI * pow(rad, 2);
44     return area;
45 } //end getArea function
46
47 double getDiameter(double rad)
48 {
49     return 2 * rad;
50 } //end getDiameter function
```

Figure 9-26 Code for Lab 9-1

Lab 9-2: Plan and Create

Problem specification

Many car dealers offer customers a choice of either a large cash rebate or an extremely low financing rate that is much lower than a local credit union charges. Create a program that calculates the monthly car payment for each of the following two scenarios:

- Scenario 1: The user accepts the dealer's rebate offer and finances the car through his or her local credit union.
- Scenario 2: The user declines the dealer's rebate offer but accepts the dealer's lower financing rate.

The formula for calculating a periodic payment on a loan is shown below. In the formula, *principal* is the amount of the loan, *rate* is the periodic interest rate, and *term* is the number of periodic payments. Also shown below are two examples that use the formula to calculate a periodic payment. Example 1 calculates the annual payment for a \$9,000 loan for three years at 5% interest. The annual payment rounded to the nearest cent is \$3,304.88. In other words, if you borrow \$9,000 for three years at 5% interest, you would need to make three annual payments of \$3,304.88 to pay off the loan. Example 2 calculates the monthly payment for a \$12,000 loan for five years at 6% interest. To pay off this loan, you would need to make 60 payments of \$231.99.

When you apply for a loan, the lender typically quotes you an annual interest rate and expresses the term in years. Therefore, when calculating a monthly payment, you must convert the annual interest rate to a monthly rate by dividing the annual rate by 12. You also need to convert the term from years to months by multiplying the number of years by 12.

Periodic payment formula

$\text{principal} * \text{rate} / (1 - (\text{rate} + 1)^{-\text{term}})$

Example 1

Principal: 9,000

Annual rate: 0.05

Term (years): 3

Formula: $9,000 * 0.05 / (1 - (0.05 + 1)^{-3})$

Annual payment: \$3,304.88 (rounded to the nearest cent)

annual payment for a \$9,000 loan for 3 years at 5% interest

Figure 9-27 Problem specification for Lab 9-2

Lab 9-2: Plan and Create (cont'd.)

<u>Example 2</u>	<u>monthly payment for a \$12,000 loan for 5 years at 6% interest</u>
Principal:	12,000
Monthly rate:	0.005 (annual rate of 0.06 divided by 12)
Term (months):	60 (5 years multiplied by 12)
Formula:	$12,000 * 0.005 / (1 - (0.005 + 1)^{-60})$
Monthly payment:	\$231.99 (rounded to the nearest cent)

Figure 9-27 Problem specification for Lab 9-2

Lab 9-2: Plan and Create (cont'd.)

main function

Input

car price
rebate
credit union rate (annual)
dealer rate (annual)
term (years)

Processing

Processing items: none

Algorithm:

1. enter the car price, rebate, credit union rate, dealer rate, and term
2. call the `getPayment` function to calculate the credit union payment; pass the car price minus the rebate, the credit union rate / 12, and the term * 12
3. call the `getPayment` function to calculate the dealer payment; pass the car price, the dealer rate / 12, and the term * 12

Output

credit union payment
dealer payment

Figure 9-28 IPO chart for the main function

Lab 9-2: Plan and Create (cont'd.)

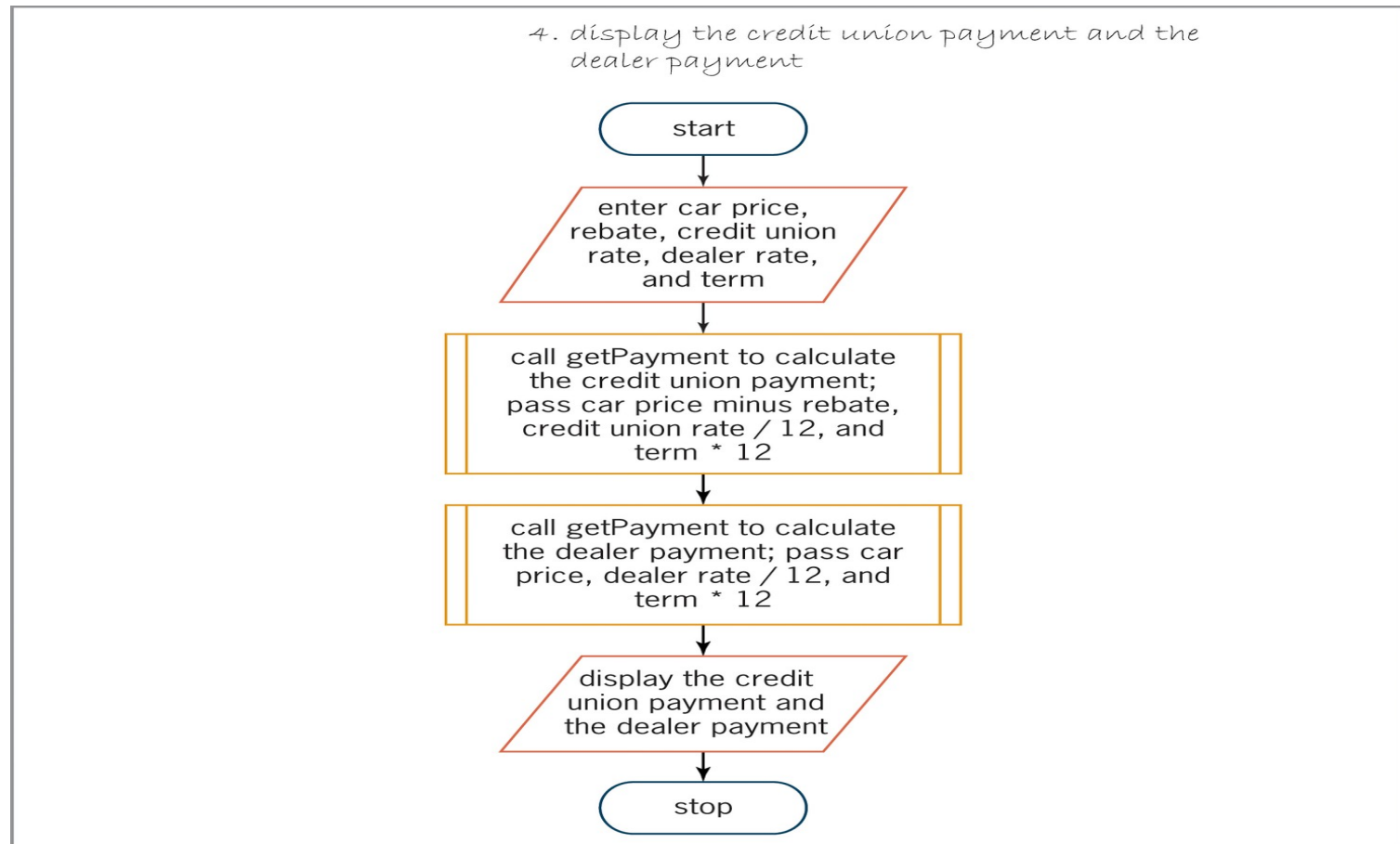


Figure 9-28 IPO chart for the main function

Lab 9-2: Plan and Create (cont'd.)

getPayment function

Input

principal
monthly rate
number of months

Processing

Processing items: none

Output

monthly payment

Algorithm:

1. calculate the monthly payment
using the periodic payment formula
2. return the monthly payment

Figure 9-29 IPO chart for the getPayment function

Lab 9-2: Plan and Create (cont'd.)

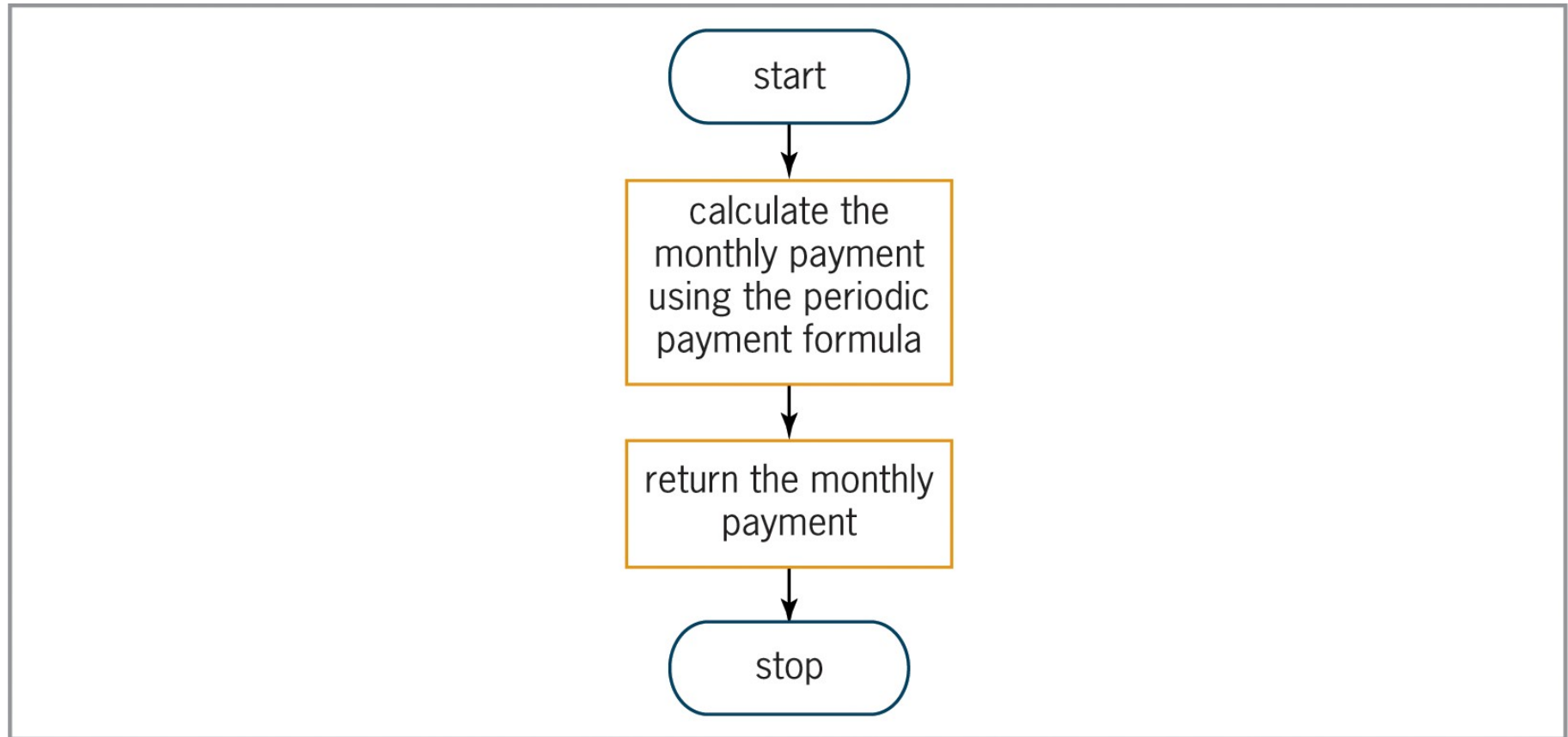


Figure 9-29 IPO chart for the `getPayment` function

Lab 9-2: Plan and Create (cont'd.)

main function IPO chart information Input car price rebate credit union rate (annual) dealer rate (annual) term (years) Processing none Output credit union payment dealer payment Algorithm enter t e car price rebate credit union rate dealer rate and term call t e et aymment unction to calculate t e credit union payment pas t e car price minus t e rebate t e credit union rate and t e term call t e et aymment unction to calculate t e dealer payment pass t e car price t e dealer rate and t e term display t e credit union payment and t e dealer payment	main function C++ instructions <pre>int carPrice = 0; int rebate = 0; double creditRate = 0.0; double dealerRate = 0.0; int term = 0; double creditPayment = 0.0; double dealerPayment = 0.0; cout << "Car price (after any trade-in): "; cin >> carPrice; cout << "Rebate: "; cin >> rebate; cout << "Credit union rate: "; cin >> creditRate; cout << "Dealer rate: "; cin >> dealerRate; cout << "Term in years: "; cin >> term; creditPayment = getPayment (carPrice - rebate, creditRate / 12, term * 12); dealerPayment = getPayment(carPrice, dealerRate / 12, term * 12); cout << "Credit union payment: \$" << creditPayment << endl; cout << "Dealer payment: \$" << dealerPayment << endl;</pre>
---	--

Figure 9-31 IPO chart information and C++ instructions for the main function

Lab 9-2: Plan and Create (cont'd.)

getPayment function

IPO chart information

Input

principal (formal parameter)
monthly rate (formal parameter)
number of months (formal parameter)

Processing

none

Output

monthly payment

Algorithm

calculate the monthly payment using
the periodic payment formula
return the monthly payment

getPayment function

C++ instructions

```
int prin  
double monthRate  
int months
```

```
double monthPay = 0.0;
```

```
monthPay = prin * monthRate /  
            (1 - pow(monthRate + 1, -months));  
return monthPay;
```

Figure 9-32 IPO chart information and C++ instructions
for the getPayment function

Lab 9-2: Plan and Create (cont'd.)

```
1 //Lab9-2.cpp - displays two monthly car payments
2 //Created/revised by <your name> on <current date>
3
4 #include <iostream>
5 #include <cmath>
6 #include <iomanip>
7 using namespace std;
8
9 //function prototype
10 double getPayment(int, double, int);
11
12 int main()
13 {
14     int carPrice = 0;
15     int rebate = 0;
16     double creditRate = 0.0;
17     double dealerRate = 0.0;
18     int term = 0;
19     double creditPayment = 0.0;
20     double dealerPayment = 0.0;
21
22     cout << "Car price (after any trade-in): ";
23     cin >> carPrice;
24     cout << "Rebate: ";
25     cin >> rebate;
26     cout << "Credit union rate: ";
27     cin >> creditRate;
28     cout << "Dealer rate: ";
29     cin >> dealerRate;
30     cout << "Term in years: ";
31     cin >> term;
32
33     //call function to calculate payments
34     creditPayment = getPayment(carPrice - rebate,
35                               creditRate / 12, term * 12);
36     dealerPayment = getPayment(carPrice,
37                               dealerRate / 12, term * 12);
38
39     //display payments
40     cout << fixed << setprecision(2) << endl;
41     cout << "Credit union payment: $"
42           << creditPayment << endl;
43     cout << "Dealer payment: $"
44           << dealerPayment << endl;
45     return 0;
}
```

the names
of the formal
parameters are
not required

Figure 9-34 Beginning of finalized Car Payment Program

Lab 9-2: Plan and Create (cont'd.)

```
46 }//end of main function
47
48 /**function definitions**/
49 double getPayment(int prin,
50                  double monthRate,
51                  int months)
52 {
53     //calculates and returns a monthly payment
54     double monthPay = 0.0;
55     monthPay = prin * monthRate /
56         (1 - pow(monthRate + 1, -months));
57     return monthPay;
58 } //end of getPayment function
```

Figure 9-34 Completion of finalized Car Payment Program

Lab 9-3: Modify

- Modify the program in Lab9-2.cpp. Make the changes listed in Figure 9-35 below. Save the modified program as Lab9-3.cpp
 - Save, run, and test the program.
1. Compare both monthly payments and then display one of the following three messages:
 - a. Take the rebate and finance through the credit union.
 - b. Don't take the rebate. Finance through the dealer.
 - c. You can finance through the dealer or the credit union.
 2. The user should be able to calculate the monthly payments as many times as needed without having to run the program again.
 3. Allow the user to enter an interest rate as either a whole number or a decimal number. For example, if the interest rate is 5%, the user should be able to enter the rate as either 5 or .05.

Figure 9-35 Modifications for Lab 9-3

Lab 9-4: What's Missing?

- The program is this lab should display the total due, given the quantity purchased and the item price.
- Follow the instructions for starting C++ and opening the Lab9-4.cpp file. Put the C++ instructions in the proper order, and then determine the one or more missing instructions.
- Test the program appropriately.

Lab 9-5: Desk-Check

- Use the data shown in Figure 9-36 to desk-check the figure's code. What current total will the code display on the screen?

Lab 9-5: Desk-Check (cont'd.)

Test data: 100 (the beginning number), 4, -3, 10, 9, and -5

```
1 //Lab9-5.cpp - displays the current total
2 //Created/revise by <your name> on <current date>
3
4 #include <iostream>
5 #include <cmath>
6 using namespace std;
7
8 //function prototype
9 double getCurrentTotal(int current, int num);
10
11 int main()
12 {
13     int currentTotal = 0;
14     int number = 0;
15
16     cout << "Beginning number: ";
17     cin >> currentTotal;
18
19     for (int x = 1; x < 6; x += 1)
20     {
21         cout << "Enter a positive or negative number: ";
22         cin >> number;
23         currentTotal = getCurrentTotal(currentTotal, number);
24     } //end for
25
26     cout << endl << "Current total: " << currentTotal << endl;
27     return 0;
28 } //end of main function
29
30 //*****function definitions*****
31 double getCurrentTotal(int current, int num)
32 {
33     if (num >= 0)
34         current += pow(num, 2);
35     else
36         current += num * 2;
37     //end if
38     return current;
39 } //end of getCurrentTotal function
```

Figure 9-36 Test data and code for Lab 9-5

Lab 9-6: Debug

- Test the program in the Lab9-6.cpp file using the data 20500, 3500, and 10 as the asset cost, salvage value, and useful life, respectively.
- The depreciation should be \$1700.00
- Debug the program